MERN STACK

TRAINING TR-103 REPORT DAY 31

30 July 2026

Overview:

Day 31 of the MERN Stack training focused on developing the Login API for the CareerConnect – Career Management System. I learned how to authenticate registered users by verifying their email and password with the data stored in the MongoDB database. The Login API was developed using Express.js with a POST request. I also learned how to return user details after successful authentication and handle invalid login credentials using appropriate error responses.

Learning Objectives:

- Understand the working of a Login API.
- Learn user authentication using Express.js.
- Receive login credentials through a POST request.
- Retrieve user information from MongoDB.
- Verify email and password.
- Return successful authentication responses.
- Handle invalid login credentials.
- Implement error handling using `try-catch`.

Project: CarrerConnect – Carrer Guidance and Management System

Login API Development

The Login API is developed to authenticate registered users in the CareerConnect career management system.

• It verifies the email and password entered by the user with the data stored in the MongoDB database.

• The API is created using Express.js with a POST request.

API Endpoint:

```
POST /login
```

• The user email and password are received from the frontend using req.body.

• The API extracts the login credentials:

1. Email
2. Password

• The system searches for a matching user in the database using:

```
User.findOne({ email, password })
```

• If matching user details are found, authentication is successful.

• The server returns the user's details such as:

1. User ID
2. User Name
3. User Type

• If the credentials are incorrect, the API returns:

```
Invalid Email or Password
```

• Error handling is implemented using try-catch blocks to handle server-side issues.

Working

1. The app.post("/login") route creates the Login API.
2. User login credentials are received from the frontend.
3. The email and password are extracted from the request body.
4. The database is searched using the entered credentials.
5. If a matching user is found, authentication is completed successfully.
6. User information is returned to the frontend.
7. The frontend uses the received usertype for role-based navigation.
8. If no matching user is found, an appropriate error message is returned.

Conclusion:

By the end of Day 31, I gained a clear understanding of developing a Login API using Express.js and MongoDB. I learned how to receive login credentials through a POST request, verify the user's email and password, retrieve user information from MongoDB, and return appropriate success or error responses. I also understood how the returned `usertype` can be used for role-based navigation, strengthening my understanding of user authentication, backend API development, MongoDB operations, and error handling in MERN Stack applications.



```
app.post("/login",async(req,res)=>{
    try{
        const {email,password}=req.body;

        const user=await User.findOne({
            email,
            password
        });

        if(!user)
            return res.json({
                success:false,
                message:"Invalid Email or Password"
            });

        res.json({
            success:true,
            user
        });

    }catch(err){
        res.json({
            success:false,
            message:err.message
        });
    }
});
```